

SNE/t-SNE and Anomaly Detection in APOGEE Spectra

A Course Project for
EP4130/PH6130 Data Science and Analysis

Done by

Akshay S Harits

ES23BTECH11002

Ulaganathan K B

ES23BTECH11039

A Report Submitted to
Indian Institute of Technology Hyderabad



Department of Physics

Course Instructor

Dr. Shantanu Desai

Contents

Abstract	iii
1 Introduction	1
1.1 The Curse of Dimensionality	1
1.2 Big Data in Astronomy	2
1.3 Report Structure	2
2 SNE, t-SNE, and the Testbed	3
2.1 Why PCA Is Not Enough	3
2.2 SNE: Replacing Distance with Neighborhood Probability	4
2.3 But There Are Some Issues	5
2.3.1 Why Early Exaggeration Matters So Much	6
2.4 How t-SNE Changes the Setup	6
2.5 Perplexity, Degrees of Freedom, and Kernel Choice	8
2.5.1 Perplexity in the Student's t Setting	9
2.5.2 Varying the Degrees of Freedom	10
2.5.3 A Power-Law Alternative	10
2.6 Our Custom Autograd Engine	11
2.7 Metrics Used	12
2.8 Barnes–Hut t-SNE on the Full MNIST Dataset	12
3 APOGEE: From Spectra to Outliers	14
3.1 Data and Workflow	14
3.2 Selecting the Sample	15
3.3 Preprocessing	15
3.4 PCA Compression	16
3.5 The t-SNE Map	17

3.6	DBSCAN Outliers	19
3.7	Outliers in Detail	20
3.8	A Brief GALAH Trial	22
	Conclusion	25
	References	25
A	Selected Code Snippets	27
A.1	Core SNE/t-SNE Code	27
A.2	APOGEE Processing Pipeline	30
A.3	Outlier Detection and Follow-up	33

Abstract

High-dimensional data is difficult to analyze because the geometry becomes unreliable. In spaces with thousands of features, Euclidean distance stops behaving the way our intuition expects, and methods that work well in low dimensions begin to fail. This report addresses that problem through a two-part unsupervised learning pipeline built for high-dimensional stellar spectra.

In the first part, our main goal was to study and understand the behavior of Stochastic Neighbor Embedding (SNE) and t-Distributed Stochastic Neighbor Embedding (t-SNE) in a direct and experimental way. To do that properly, we built a custom automatic differentiation engine as a tool, not as an end in itself. This gave us the freedom to test how the embedding changes under different similarity distributions and hyperparameters, especially perplexity, degrees of freedom, and early exaggeration. That part of the work was about understanding why these choices matter and how they shape the final low-dimensional map.

In the second part, we applied this framework to 15,372 high-resolution infrared spectra from the SDSS APOGEE DR17 `allStar-dr17-synspec_rev1.fits` catalog. After pre-processing, masking, and normalization, we reduced the data with Principal Component Analysis (PCA) to 50 components while retaining 90.01% of the variance. We then used t-SNE for dimensionality reduction and DBSCAN for clustering in order to identify unusual structure in the embedded space. The final pipeline isolated one dominant stellar population and 25 clear astrophysical anomalies. Cross-checks with SIMBAD showed that these outliers correspond to physically meaningful objects, including young stellar objects, peculiar Be stars, and high proper-motion dwarfs.

The complete implementation used in this report is available at https://github.com/AkshayHarits/tsne_project/tree/add-phase2.

1. Introduction

This report starts from a simple question: what should we do when the data are too large and too high-dimensional to understand directly? In two or three dimensions, we can inspect patterns by plotting points and measuring distances. In thousands of dimensions, that intuition no longer helps. The geometry changes, the distances stop being informative, and even familiar algorithms begin to behave poorly.

Our goal is to keep the story simple. First, we explain why high-dimensional data are difficult in principle. Then we show how SNE, later t-SNE, and a few custom ideas built around the same neighborhood-based view help address that difficulty. Finally, we apply that framework to stellar spectra, where each star is described not by a few measurements but by thousands of flux values.

1.1 The Curse of Dimensionality

The phrase **Curse of Dimensionality** describes what happens when the number of features becomes very large. As dimensions increase, the space grows so quickly that the available data occupy only a tiny fraction of it. Even if the dataset looks large in ordinary terms, it becomes sparse relative to the space it lives in.

That sparsity creates a practical problem. In low dimensions, nearby points are meaningfully nearby, and distant points are clearly different. In very high dimensions, that contrast weakens. Points begin to look similarly far apart, and the idea of a clean geometric neighborhood starts to break down. This is why many standard tools lose reliability when the feature count becomes extreme.

Three ideas are especially important here:

- **Data sparsity:** Adding more features does not automatically make a model better. After a point, extra dimensions reduce data density, amplify noise, and make overfitting more likely.
- **Volume mismatch:** High-dimensional geometry behaves in ways that are hard to picture. Regions that seem large in low dimensions may contain very little useful volume, while most of the space sits far from the center.
- **Distance concentration:** As dimension increases, the gap between the nearest and farthest neighbors shrinks. When almost every point is at a similar distance, Euclidean distance stops being a reliable measure of similarity.

This is the key transition for the rest of the report. If raw distance is no longer trust-

worthy, then we need a different language for similarity. Instead of treating distance as absolute, methods such as SNE and t-SNE describe neighborhoods in probabilistic terms. That shift makes it possible to preserve local structure even when the original space is too large to visualize directly.

1.2 Big Data in Astronomy

Astronomy is a natural setting for this problem because modern surveys generate data at a scale that no human can inspect manually. Telescopes now act as measurement systems and data pipelines at the same time, producing catalogs large enough that automatic methods are no longer optional.

For this project, we worked with the **Sloan Digital Sky Survey (SDSS)**, specifically the **APOGEE DR17** release. APOGEE records high-resolution near-infrared H-band spectra for stars across the Milky Way. From the 4GB `allStar-dr17-synspec_rev1.fits` catalog, we extracted 15,372 raw targets.

A single spectrum here is not a short vector of summary values. It is a sequence of 8,575 flux measurements. That means each star lives in an 8,575-dimensional space. In such a setting, rare objects cannot be found reliably by eye or with a simple plot. We need an unsupervised pipeline that can reduce the dimensionality, preserve the local structure, and expose unusual objects without relying on manual labels.

1.3 Report Structure

As mentioned in the abstract, the report has two connected parts. The first part focuses on understanding SNE, t-SNE, and the behavior of the embedding under different design choices and hyperparameters. The second part applies that framework to the APOGEE spectra for dimensionality reduction, clustering, and anomaly discovery.

We begin with the first part and build the mathematical story of SNE/t-SNE step by step and then delve into the APOGEE spectra.

2. SNE, t-SNE, and the Testbed

We start with a fair question: if Principal Component Analysis (PCA) already exists, why not just use PCA and move on? PCA is fast, clean, and very useful. But PCA is a linear method. It finds directions of maximum variance, not directions that necessarily preserve the true neighborhood structure of a non-linear dataset. For many real datasets, especially images and spectra, that difference matters.

In this part of the project, our goal was to understand that difference carefully. We wanted to see where PCA helps, where it fails, how SNE improves the situation, and why t-SNE goes one step further. This is also where our custom autograd engine enters the story: not as the final objective, but as the tool that let us experiment with kernels, hyperparameters, and optimization behavior in a controlled way.

2.1 Why PCA Is Not Enough

PCA is the natural first baseline because it reduces dimensionality by projecting the data onto directions of maximum variance. That makes it excellent for compression and for simple linear structure. The problem is that preserving variance is not the same thing as preserving neighborhoods. If the true geometry of the data lies on a curved or folded manifold, a linear projection can flatten different groups into the same region of the map.

The MNIST digits dataset shows this clearly. Digits that are visually different may still overlap after a linear projection because PCA is not trying to separate classes or preserve local neighborhoods. It is trying to explain variance with a small number of global directions. That is useful, but it is not enough for the kind of structure we want to study here.

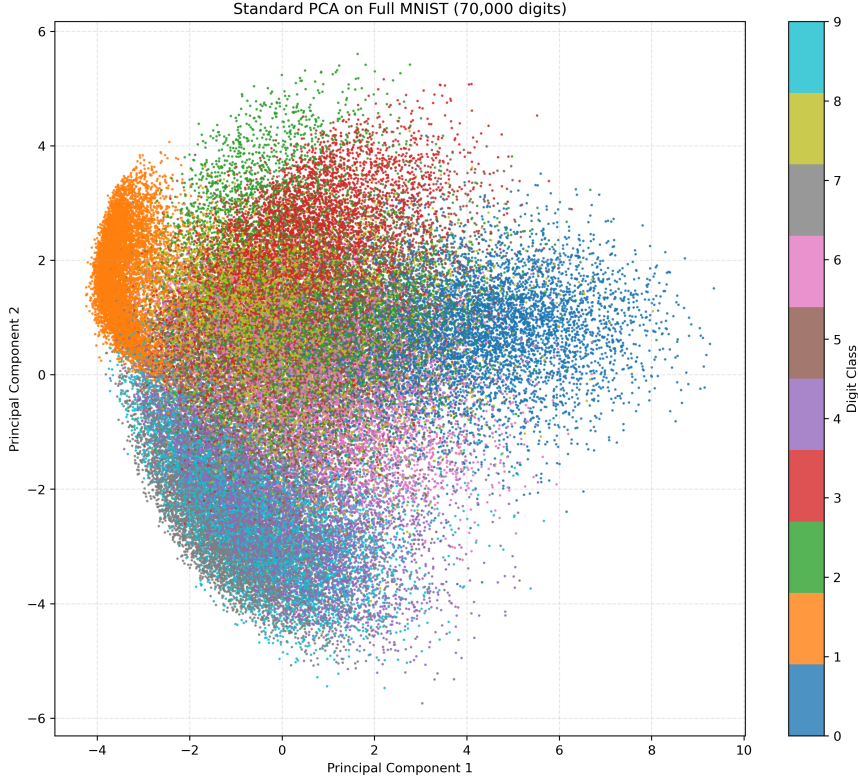


Figure 2.1: Standard PCA on the full MNIST dataset. The classes overlap heavily, so the projection is hard to interpret as a meaningful neighborhood-preserving map.

This figure makes the limitation very clear. We do see broad variance directions, but the map does not separate the structure in a way that is easy to trust or interpret. Too many classes smear into one another, and the geometry is not telling a clean story. That is why we move beyond PCA. We are not only asking for a lower-dimensional representation. We are asking for one that keeps nearby points meaningfully nearby.

2.2 SNE: Replacing Distance with Neighborhood Probability

SNE starts from a simple but powerful idea. Instead of trusting raw Euclidean distance directly, it converts distances into probabilities. In the original high-dimensional space, this similarity is modeled using a Gaussian centered at each point. For each point x_i , SNE defines a conditional probability distribution over all other points:

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}, \quad p_{i|i} = 0.$$

If x_j is close to x_i , then $p_{j|i}$ is large. If it is far away, the probability is small. Note that $p_{i|i} = 0$ (a point cannot be its own neighbor, preventing trivial self-similarity). So the problem is no longer framed as preserving exact distances. It becomes a problem of preserving neighborhood relationships.

The bandwidth σ_i is not fixed globally. It is chosen using the perplexity, which can be read as the effective number of neighbors around each point:

$$\text{Perp}(P_i) = 2^{H(P_i)}.$$

This is one of the ideas that makes SNE flexible. Dense regions can use a smaller local scale, while sparse regions can use a larger one.

In the low-dimensional map, SNE again uses a Gaussian model, but now with a fixed variance. Following the original formulation, the variance is set to $1/2$, which is equivalent to using a Gaussian with standard deviation $1/\sqrt{2}$. The low-dimensional conditional probability is therefore

$$q_{j|i} = \frac{\exp(-\|y_i - y_j\|^2)}{\sum_{k \neq i} \exp(-\|y_i - y_k\|^2)}, \quad q_{i|i} = 0.$$

This Gaussian choice in the map is not inevitable. It is a modeling decision, and later we will change it. That change is exactly what gives t-SNE its main advantage.

SNE then minimizes the mismatch between the original and mapped neighborhoods using KL divergence:

$$C_{\text{SNE}} = \sum_i \text{KL}(P_i \| Q_i).$$

This is the conceptual shift that matters most: SNE does not ask, “Can I keep all distances exact?” It asks, “Can I keep the neighborhood story intact?”

2.3 But There Are Some Issues

SNE is already a major improvement over PCA for non-linear structure, but it still runs into two practical problems.

The first is the **crowding problem**. In high dimensions, there is enough room to place many moderately distant neighbors around a point. In two dimensions, there is not. So when we try to preserve many local relationships at once, those moderate-distance points get squeezed into a small area of the map. The result is a cluttered and hard-to-read embedding.

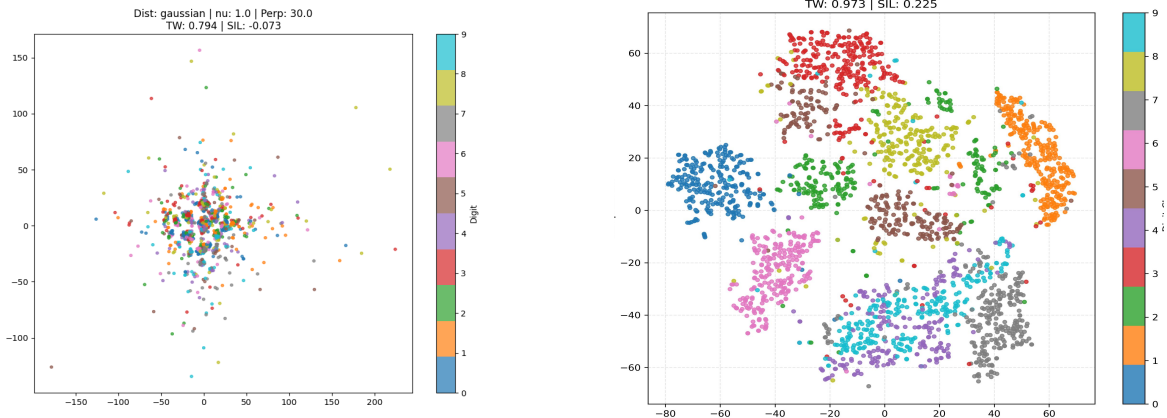
The second problem is that the optimization is not especially friendly. The original SNE formulation works with asymmetric conditional probabilities, which makes the objective harder to optimize and the geometry of the gradient less balanced. In the original line of work, this led to practical tricks such as momentum, simulated annealing through

injected noise, and later variants such as symmetric SNE and UNI-SNE that tried to improve either the optimization or the crowding behavior.

Our Gaussian low-dimensional run shows this failure mode clearly. The points collapse into a dense central mass with weak class separation. Even before looking at the labels, the map feels crowded. The two summary metrics shown in the plot tell the same story: trustworthiness is only moderate, and the silhouette score is slightly negative, which means the groups are still strongly overlapping.

2.3.1 Why Early Exaggeration Matters So Much

In our own experiments, one very important practical intervention was stronger early exaggeration. While it is not the fundamental conceptual fix, it changes the early optimization dynamics enough to let real local groups form before the map settles. The comparison below shows how strongly the result changes when the exaggeration phase is made stronger.



Baseline Gaussian run: crowded map, trustworthiness = 0.794, silhouette = -0.073 .

Strong early exaggeration: clearer clusters, trustworthiness = 0.973, silhouette = 0.225.

Figure 2.2: Effect of stronger early exaggeration in the Gaussian setting. A better early attraction phase makes the map much easier to read.

Even so, this is still not the full answer. The deeper issue is the low-dimensional model itself. This is the point where the story naturally moves from SNE to t-SNE.

2.4 How t-SNE Changes the Setup

t-SNE keeps the neighborhood-based philosophy of SNE, but changes two important parts of the method.

First, it replaces the asymmetric conditional probabilities with a symmetric joint distribution in the high-dimensional space:

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}.$$

Second, it replaces the Gaussian low-dimensional kernel with a heavy-tailed Student's t distribution:

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}.$$

The cost function then becomes a single KL divergence between the joint distributions:

$$C_{\text{tSNE}} = \text{KL}(P \| Q).$$

The effect of the heavy tails is crucial. In simple terms, they give the map more room. Nearby points can still stay close, but points that should be separate are pushed apart more effectively. This is the core reason t-SNE handles the crowding problem much better. It also makes the optimization friendlier, because the low-dimensional forces behave more sensibly over long range than they do in plain SNE.

There is also an optimization advantage here. In symmetric SNE, the objective becomes cleaner, but t-SNE goes further by making the low-dimensional interaction decay more gradually with distance. This means separated groups do not immediately lose all influence on one another. If two parts of the same structure drift apart early in optimization, t-SNE still has enough long-range interaction to pull them back into a sensible global arrangement.

The gradient makes this interpretation clear:

$$\frac{\partial C}{\partial y_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij})(y_i - y_j) (1 + \|y_i - y_j\|^2)^{-1}.$$

The term $(p_{ij} - q_{ij})$ measures whether a pair is too close or too far apart in the map, while the extra factor $(1 + \|y_i - y_j\|^2)^{-1}$ comes from the heavy-tailed kernel. Intuitively, nearby true neighbors still attract strongly, but distant mismatched points repel in a way that remains useful instead of collapsing into the same dense center. This is exactly the kind of behavior we want before looking at the actual embedding.

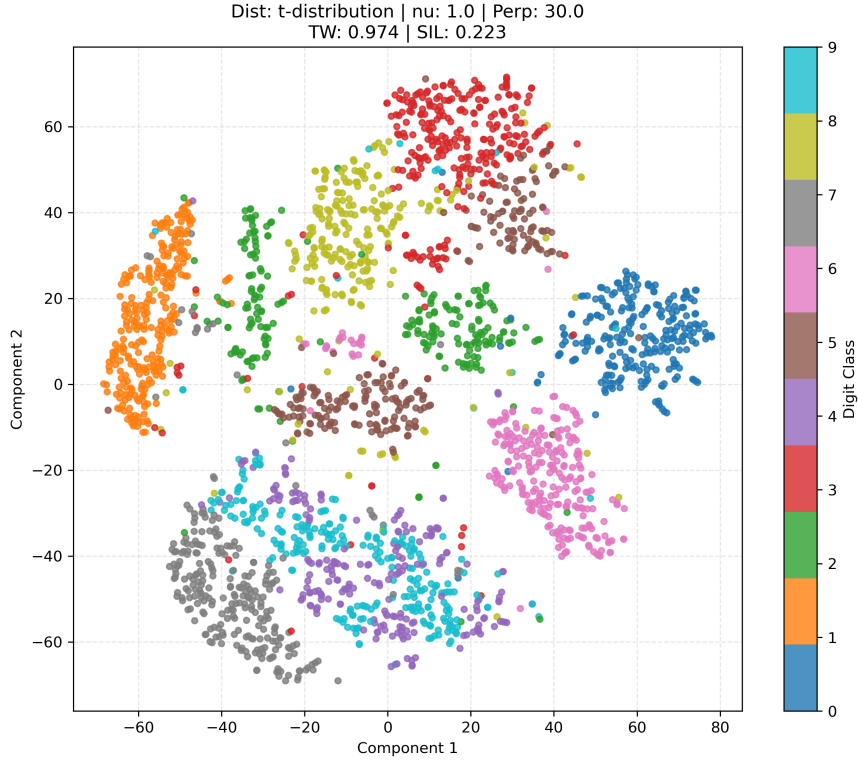


Figure 2.3: A t-distribution low-dimensional embedding with $\nu = 1.0$ and perplexity 30. On the same 2,500-sample MNIST subset, the map is far clearer and the classes separate into meaningful local groups.

Note: Figure 2.1 uses the full 70,000-image MNIST dataset, but Figures 2.2 and 2.3 use a 2,500-sample subset. This is a practical limitation of our custom implementation, which is much slower at that scale. Standard large-scale t-SNE implementations such as the one in scikit-learn use the Barnes-Hut approximation, which is much faster for bigger datasets.

2.5 Perplexity, Degrees of Freedom, and Kernel Choice

After establishing why t-SNE improves on SNE, we used our testbed to study how the embedding changes as we vary the main hyperparameters. Rather than looking at a single final plot, this section treats the embedding as a dynamical object: we change one design choice at a time and observe how the geometry responds.

All figures in this section are generated on the same 2,500-sample MNIST subset used for the custom low-dimensional experiments. This keeps the comparisons fair and keeps the runtime manageable for our exact implementation.

2.5.1 Perplexity in the Student's t Setting

We begin by fixing the low-dimensional kernel to the standard Student's t form with $\nu = 1.0$ and varying the perplexity. This is the cleanest way to see how the effective neighborhood size changes the map.

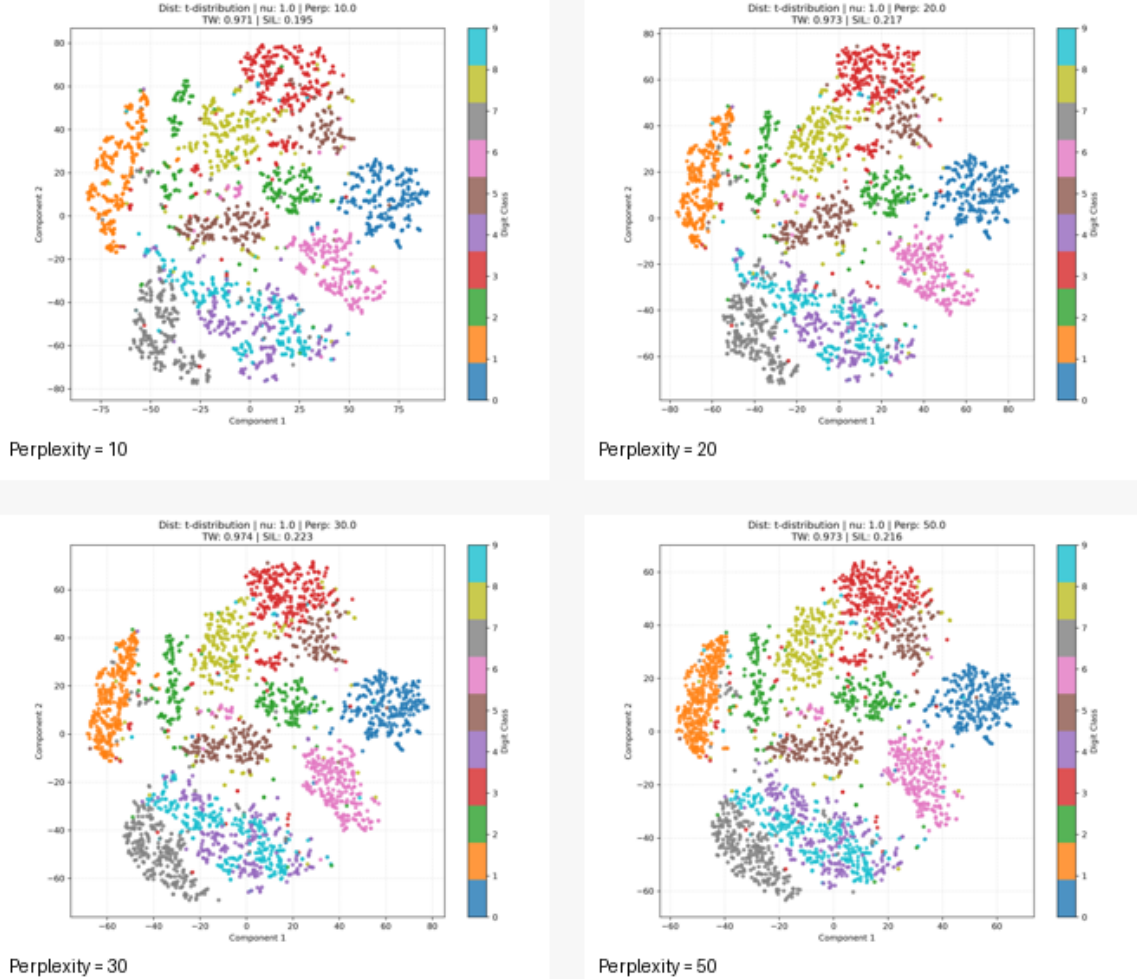


Figure 2.4: t-distribution embeddings with fixed $\nu = 1.0$ and varying perplexity. Lower perplexity emphasizes very local structure, while higher perplexity produces smoother, more global organization.

A clear trend appears across the grid. At perplexity 10, the map is strongly local: clusters separate, but several groups look more fragmented and slightly over-responsive to local density. At perplexity 20 and 30, the embedding becomes more balanced. The classes remain well separated, but the internal geometry of each cluster becomes more stable and readable. By perplexity 50, the map is smoother and more globally organized, but some fine local separation begins to soften. For this dataset and this sample size, the middle range around 20–30 gives the best balance between local faithfulness and global clarity.

2.5.2 Varying the Degrees of Freedom

Next, we fix the perplexity at 30 and vary the degrees of freedom parameter ν in the Student’s t kernel. This lets us directly study the effect of tail thickness in the low-dimensional model.

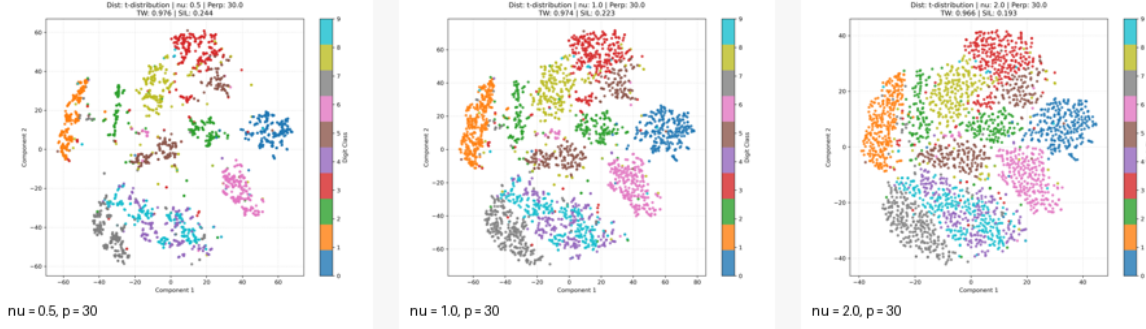


Figure 2.5: t-distribution embeddings at fixed perplexity 30 and varying degrees of freedom. Changing ν changes the tail thickness and therefore the strength of long-range repulsion between clusters.

Again the trend is visible. With $\nu = 0.5$, the tails are heavier, and the map pushes groups apart more aggressively. This produces stronger gaps, but also a slightly stretched and less stable layout. With $\nu = 1.0$, the separation remains strong while the cluster geometry becomes more coherent. With $\nu = 2.0$, the tails become lighter, and the map compresses somewhat; the classes are still distinguishable, but the embedding looks less spacious and slightly more crowded. In our experiments, $\nu = 1.0$ consistently gave the best visual balance, which aligns with the standard t-SNE choice.

2.5.3 A Power-Law Alternative

We also tested a power-law family in the low-dimensional space. This was important for the project because our goal was not only to reproduce standard t-SNE, but to explore whether other heavy-tailed similarity models could produce useful embeddings as well.

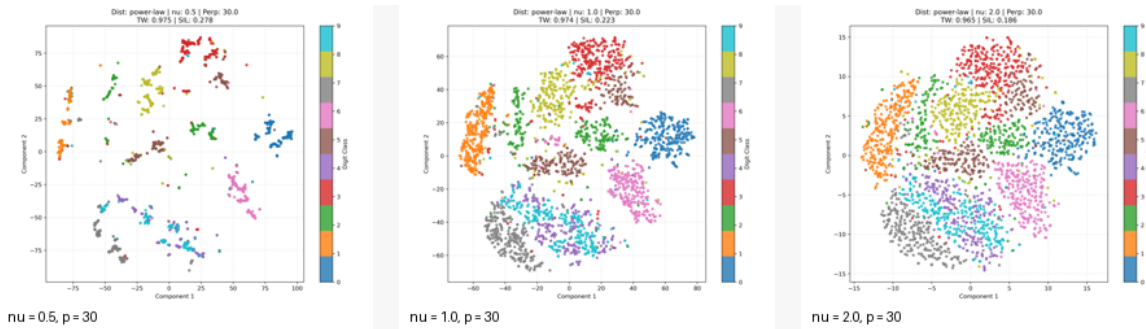


Figure 2.6: Power-law embeddings at fixed perplexity 30 and varying degrees of freedom. The kernel can work well, but its behavior is more sensitive to the choice of ν .

The power-law results are informative. At $\nu = 0.5$, the embedding becomes too sparse and unstable: many points are pushed apart too strongly, leaving excessive empty space and weaker cluster cohesion. At $\nu = 1.0$, the structure improves dramatically and becomes competitive with the Student’s t results. At $\nu = 2.0$, the map becomes more compact and still remains readable, though the cluster spacing is not quite as natural as in the best t -distribution case. Overall, the power-law family is clearly viable, but it is less forgiving than the Student’s t kernel and requires more careful tuning.

Taken together, these sweeps support three practical conclusions. First, perplexity mainly controls the local-versus-global balance of the map. Second, the degrees of freedom parameter controls how strongly clusters repel at long range. Third, while alternative heavy-tailed kernels can work, the Student’s t distribution remains the most stable and reliable choice across the range of settings we tested. For that reason, it remains the best reference point for the rest of the report, while the power-law results serve as evidence that the custom testbed can meaningfully explore beyond the standard formulation.

2.6 Our Custom Autograd Engine

One important tool behind all of these experiments was a custom automatic differentiation engine. We needed it because the project was not just about running standard t-SNE once. We wanted to modify the objective, swap low-dimensional kernels, vary degrees of freedom, and study how those choices changed the optimization.

The engine is built around a `Value` class that stores both data and gradients and supports the operations needed by the embedding pipeline: arithmetic, broadcasting-aware gradient reduction, matrix multiplication, transpose, exponentials, logarithms, sums, and means. As the loss is assembled, these operations build a computation graph. Backpropagation then traverses that graph in reverse to compute exact gradients for the custom objective.

That is what made the kernel experiments in this chapter possible. It gave us direct control over the objective and let us test Gaussian, Student’s t , and power-law formulations in a consistent way instead of treating the optimizer as a black box.

It is also the main reason the custom runs were restricted to 2,500 points. Our implementation computes exact pairwise interactions, so both runtime and memory grow quickly with dataset size. The code also had to explicitly clear graph references after backpropagation to avoid severe memory growth from large intermediate arrays. In short, the custom engine gave us flexibility and exact gradients, but at a computational cost that made full-scale MNIST impractical.

2.7 Metrics Used

To evaluate the embeddings, we used two complementary metrics: trustworthiness and silhouette score. Trustworthiness checks whether neighbors in the 2D map were genuine neighbors in the original high-dimensional space, while silhouette score checks whether the visible groups in the map are well separated.

Trustworthiness. This metric measures how well local neighborhoods are preserved under the embedding. It is defined by

$$T(k) = 1 - \frac{2}{nk(2n - 3k - 1)} \sum_{i=1}^n \sum_{j \in \mathcal{N}_i^k} \max(0, r(i, j) - k),$$

where n is the number of samples, k is the neighborhood size, $\mathcal{N}_i^k$ contains points that appear among the k nearest neighbors of point i in the low-dimensional map but not in the original space, and $r(i, j)$ is the rank of j around i in the original space. Its values lie between 0 and 1, and higher values are better.

Silhouette score. This metric measures how well-separated the groups are in the 2D embedding itself. For a point i , if $a(i)$ is the mean distance to points in the same cluster and $b(i)$ is the smallest mean distance to points in any other cluster, then

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}.$$

The overall silhouette score is the mean of $s(i)$ over all points and lies in the range $[-1, 1]$. Higher values indicate cleaner separation, values near 0 indicate overlap, and negative values indicate poor cluster geometry.

Taken together, these two metrics tell us whether an embedding is both faithful and readable. In this report, trustworthiness serves as the main neighborhood-preservation check, while silhouette score serves as the main cluster-separation check.

2.8 Barnes–Hut t-SNE on the Full MNIST Dataset

We also present the standard scikit-learn Barnes–Hut t-SNE on the full 70,000-image MNIST dataset. Unlike our custom exact implementation, this version is much more practical for large datasets, so it gives us a simple large-scale reference.

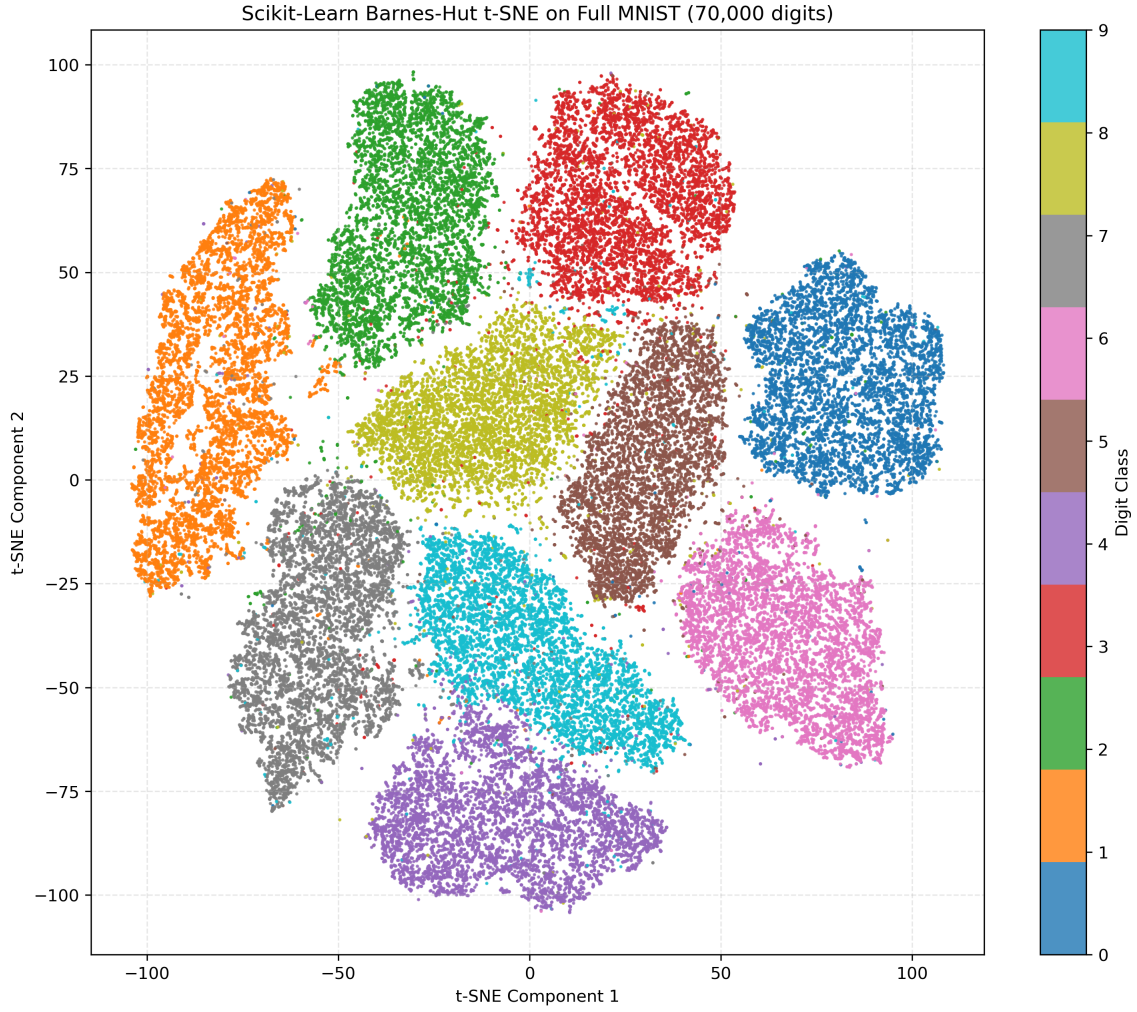


Figure 2.7: Standard scikit-learn Barnes–Hut t-SNE applied to the full 70,000-image MNIST dataset. The large-scale embedding preserves clear class regions while remaining computationally practical.

The embedding shows much clearer digit groups than PCA and is easier to read. At this point, we have understood the main ideas behind t-SNE and explored how its parameters affect the map. We now move to Part II, where we apply these ideas to a real use case: APOGEE stellar spectra for dimensionality reduction and anomaly detection.

3. APOGEE: From Spectra to Outliers

Part I was about understanding t-SNE in a controlled setting. Part II uses the same ideas on real APOGEE spectra. Two papers helped us frame this part of the project. Traven et al. (2016) showed that t-SNE can organize large collections of spectra into meaningful visual groups, while Reis et al. (2017) showed that APOGEE spectra can be searched systematically for unusual objects. We do not try to reproduce either paper exactly. Instead, we take their main lessons and build a simpler pipeline that we can understand end to end.

We should also be direct about the tone of this chapter. We are not trying to write as astrophysics experts here. Much of what we understood in this part came from reading Traven et al. (2016) and Reis et al. (2017), studying the APOGEE data products, and tracing through our own code carefully. So this chapter focuses on what we actually did, why each step is there, and what the resulting maps seem to be telling us.

3.1 Data and Workflow

We begin from the SDSS APOGEE DR17 `allStar-dr17-synspec_rev1.fits` catalog. From that master catalog, our code keeps the fields needed for the full workflow: APOGEE identifiers, telescope and field names for downloading the individual spectra, heliocentric velocity for wavelength correction, quality flags, and a few summary quantities such as effective temperature, metallicity, and surface gravity.

Those physical labels are not used to build the embedding. They are only used afterward to color the final map and check whether the structure looks physically sensible. That point matters, because it keeps the pipeline genuinely unsupervised.

The workflow itself is simple:

1. filter the DR17 catalog with basic quality cuts,
2. download the corresponding `aspcapStar` spectra,
3. preprocess the spectra so that they can be compared fairly,
4. reduce them with PCA from 8,575 flux values to 50 components,
5. run t-SNE with perplexity 100,
6. run DBSCAN on the 2D map,
7. and finally cross-check the isolated objects with SIMBAD.

This is much simpler than the pipelines in Traven et al. (2016) and Reis et al. (2017), but that is intentional. Our aim is not maximum complexity. Our aim is a clean and readable path from raw spectra to a small set of interesting outliers.

3.2 Selecting the Sample

Before dimensionality reduction begins, we first restrict the catalog to a cleaner working sample. In the catalog filtering code, we keep stars with signal-to-noise ratio greater than 100, require both `STARFLAG` = 0 and `ASPCAPFLAG` = 0 (ensuring no major pipeline warnings or bad detector pixels were flagged by APOGEE), restrict to `TEFF` < 8000, and also require `VSCATTER` < 1.0. The reason is practical: if the sample is dominated by poor-quality or unstable spectra, the anomaly detector will mostly rediscover bad data instead of interesting objects.

This is very much in the spirit of Reis et al. (2017), who also emphasize the importance of working with high-quality APOGEE spectra when searching for unusual objects. After filtering, we use the telescope, field, and APOGEE identifier values to download the corresponding `aspcapStar` files from the SDSS archive. The final working sample used in this report contains 15,372 spectra.

3.3 Preprocessing

The preprocessing stage is where the survey files become machine-learning inputs. Without this step, the later stages can end up learning missing pixels, camera gaps, and wavelength shifts instead of genuine spectral variation.

The key preprocessing steps are the following:

1. **Build a master wavelength grid.** We extract a common wavelength grid from the FITS header and use it as the reference grid for every spectrum.
2. **Mask bad pixels.** We mark a pixel as bad if the reported error is too large, if the error is missing, if the flux is missing, or if the flux is non-positive.
3. **Reject badly broken spectra.** If more than 20% of a spectrum is unusable, we remove that star from the cleaned sample.
4. **De-redshift each spectrum.** Using `VHELIO_AVG`, we shift every spectrum back toward the rest frame so that similar stars are not misaligned in wavelength.
5. **Interpolate onto the common grid.** After the shift, we interpolate the valid flux values back onto the master grid, which also fills the APOGEE camera gaps in a controlled way.
6. **Normalize each spectrum.** Finally, we divide by the median flux so that the

comparison focuses more on spectral shape than on overall scale.

This stage is easy to miss when looking only at the final plots, but it is one of the most important parts of the whole pipeline. In spirit, it matches the general message behind both Traven et al. (2016) and Reis et al. (2017): before asking whether a spectrum is unusual, we first need a representation in which ordinary technical variation has already been reduced.

3.4 PCA Compression

After preprocessing, each star is still represented by 8,575 flux values. That is too large for the later stages to handle efficiently, so we use PCA as a practical compression step. Here PCA is not the final visualization method and not the main scientific result. It simply gives us a compact representation that is much easier to feed into t-SNE.

In our implementation, we mean-center the spectra and reduce them to 50 principal components. This retains 90.01% of the total variance.

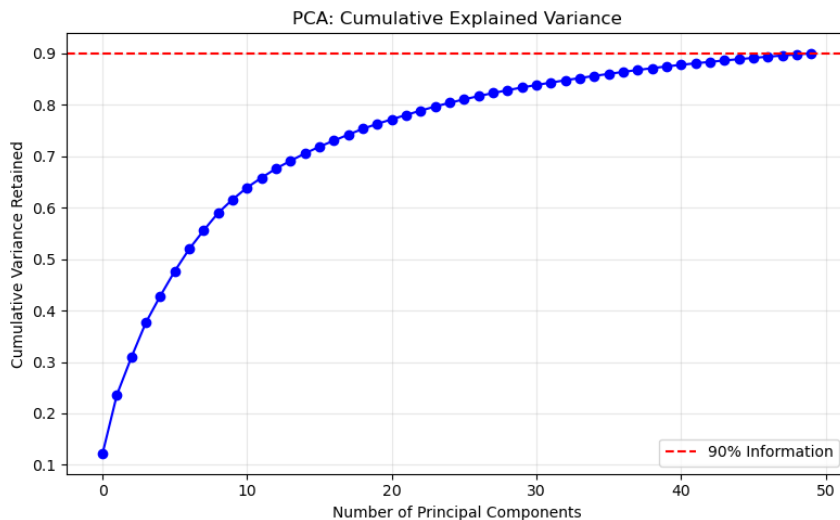


Figure 3.1: Cumulative explained variance for the APOGEE PCA step. The curve reaches 90.01% at 50 components, which is why we use that value in the rest of the pipeline.

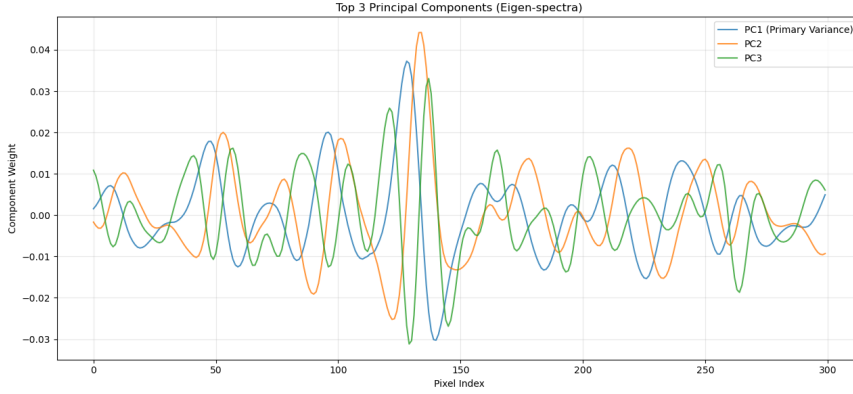


Figure 3.2: First three principal components shown as a local eigenspectrum window. The leading components clearly contain structured spectral variation rather than only random noise.

These two plots justify the PCA step in complementary ways. The cumulative variance plot explains why 50 components is a reasonable cutoff. The eigenspectra plot shows that the dominant directions of variation still look physically structured. By normalizing by median flux, we intentionally discarded absolute luminosity to focus entirely on chemical absorption shapes. Furthermore, while compressing to 50 components retains 90% of the variance, we acknowledge that the discarded 10% likely contains high-frequency noise alongside potentially rare, weak spectral lines. This is an acceptable engineering trade-off to make the manifold learning computationally feasible. So PCA acts here as a bridge between the cleaned spectra and the non-linear map that follows.

3.5 The t-SNE Map

Once the spectra are compressed to 50 dimensions, we run the standard scikit-learn t-SNE implementation with two output dimensions, perplexity 100, `max_iter = 2000`, PCA initialization, and automatic learning rate. The higher perplexity is deliberate. In Part I we worked with smaller controlled datasets; here the sample is much larger, so the effective neighborhood size also needs to be larger.

This stage is the part most directly inspired by Traven et al. (2016). Their work made it clear that t-SNE can be a useful organizer for spectral morphology. We follow that idea in a simpler way: first build the map, then ask whether known physical labels trace smooth patterns across it.

These three plots are important for interpretation. The temperature map shows a broad smooth gradient instead of random patchiness. The metallicity map suggests that some substructures are chemically distinct, especially around the narrow central branch. The surface-gravity map shows that the lower central arc is separated from other parts of the map in a physically interpretable way. None of this proves that every global shape in the embedding has a strict astrophysical meaning, and we should still be cautious there.

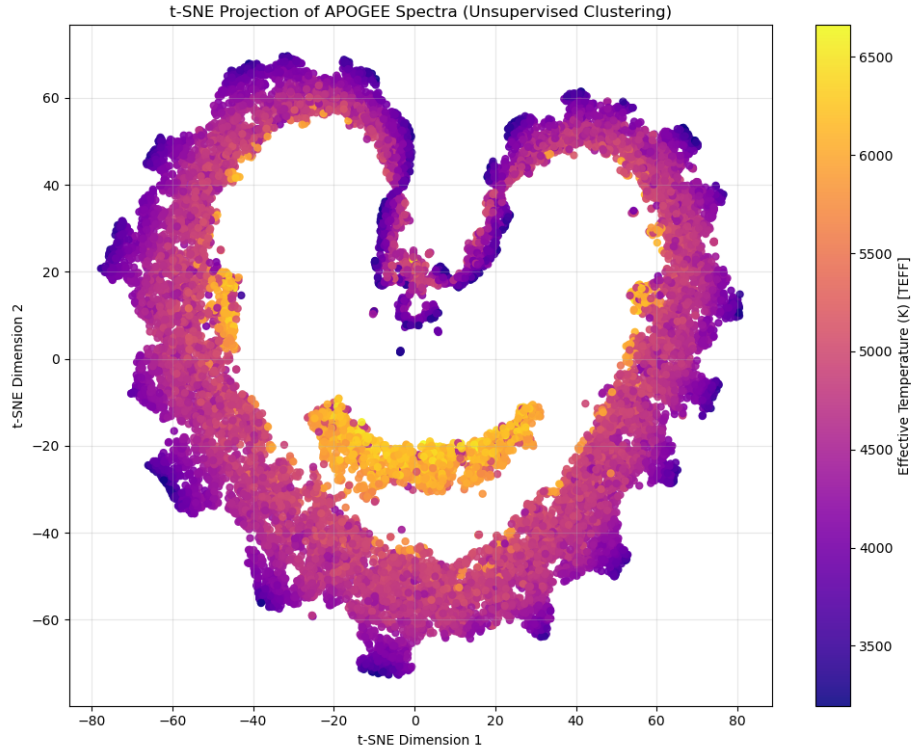


Figure 3.3: t-SNE embedding of the APOGEE spectra colored by effective temperature. The map is built without labels, but a broad temperature gradient still appears afterward.

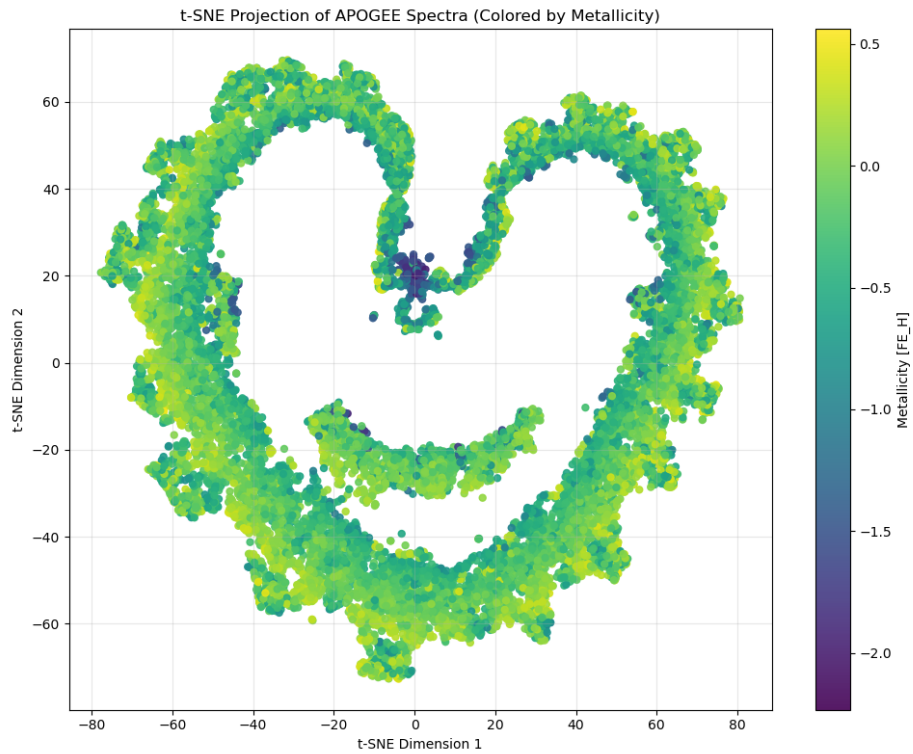


Figure 3.4: The same t-SNE embedding colored by metallicity. A narrow central structure appears notably more metal-poor than much of the surrounding population, which is a useful sign that the embedding is organizing real spectral differences.

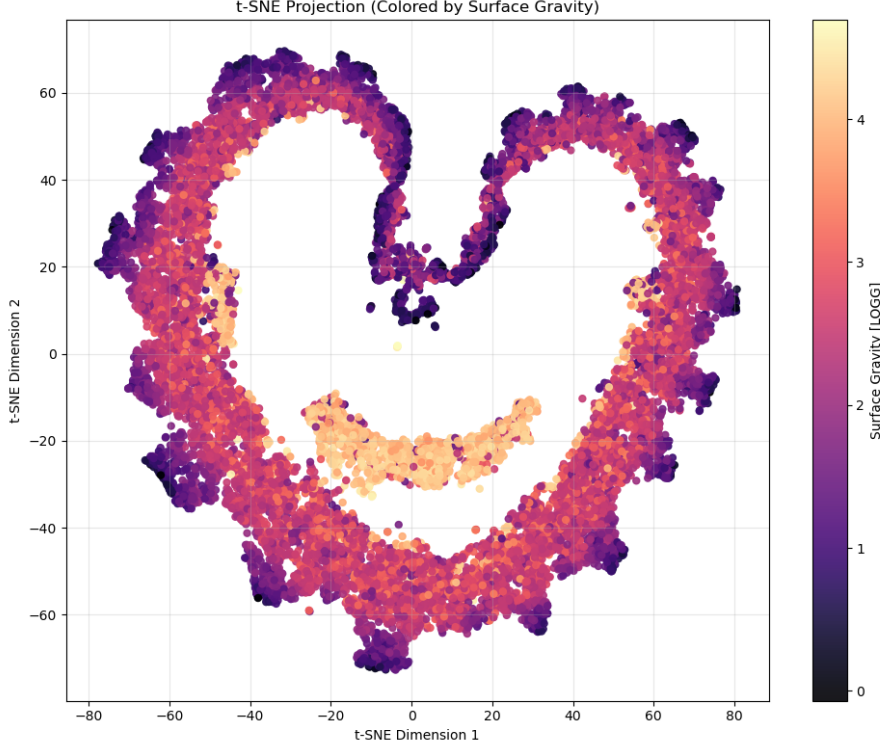


Figure 3.5: The same t-SNE embedding colored by surface gravity. The lower central arc stands out at higher LOGG, while other parts of the map sit at lower values, again suggesting that the embedding reflects meaningful physical variation.

But together these plots strongly suggest that the t-SNE map is preserving real local structure.

3.6 DBSCAN Outliers

Once the two-dimensional map is available, we switch from visualization to anomaly extraction. For this step, we use DBSCAN with `eps = 3.5` and `min_samples = 15`. The reason is simple: DBSCAN does not force every point into a cluster. Dense regions remain dense clusters, while isolated points can remain unlabeled as noise. For anomaly detection, that is exactly what we want.

This part plays a role somewhat analogous to the outlier search in Reis et al. (2017), though our method is much simpler. Their APOGEE study uses a richer similarity-based strategy. Ours is more direct: t-SNE first organizes the spectra, and DBSCAN then isolates the spectra that sit outside the dominant population.

The parameters `eps = 3.5` and `min_samples = 15` were not chosen arbitrarily. They were selected using the k-distance graph heuristic so that the density threshold isolates only a very small tail of the dataset as noise, roughly the top $\sim 0.15\%$. In practice, varying `eps` by about ± 0.5 shifts the boundary of the main cluster slightly, but the most extreme isolated points remain stable across runs. That gives us confidence that the

strongest outliers are robust anomalies and not just edge cases of the density threshold.

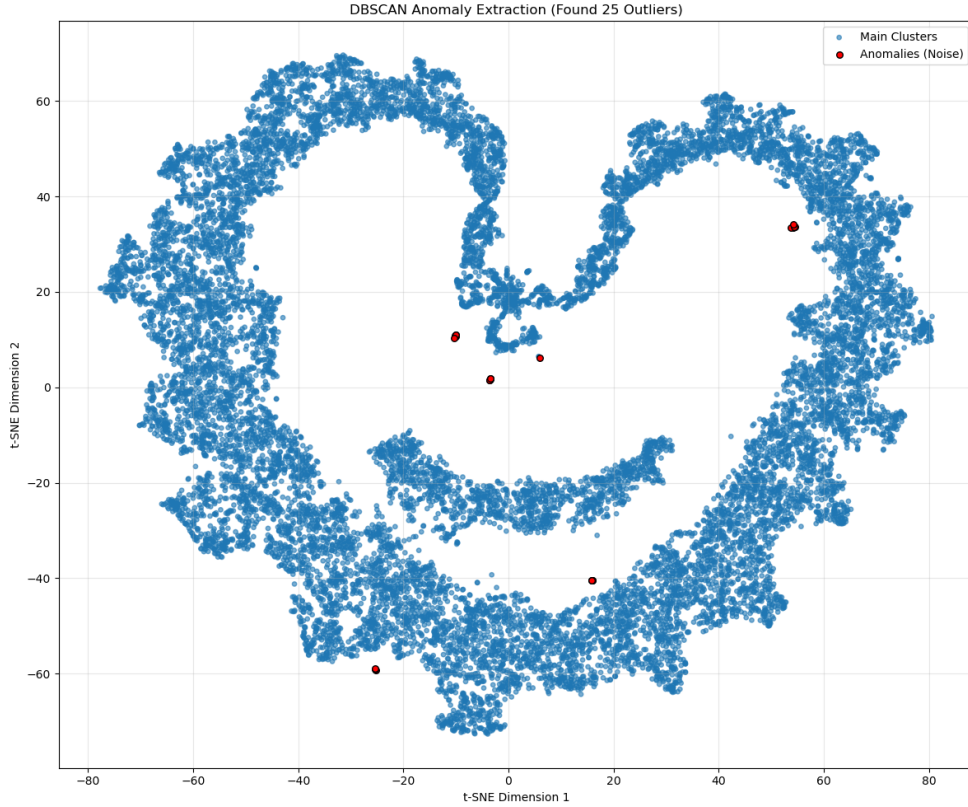


Figure 3.6: DBSCAN applied to the APOGEE t-SNE map. Most spectra belong to one dominant population, while 25 isolated points are flagged as candidate anomalies.

In the final run, DBSCAN identifies 25 outliers. That is a small enough set to inspect individually, which is what makes the result useful. The map is not broken into many tiny fragments. Instead, one dominant population remains intact, while a manageable set of isolated spectra is set aside for follow-up.

3.7 Outliers in Detail

After identifying the 25 outliers, we cross-match them against SIMBAD by converting the APOGEE identifiers into a SIMBAD-friendly `2MASS J. . .` form. We then inspect the spectra themselves. We do not use this step to make strong astrophysical claims. We use it mainly as a reality check: do these objects actually look unusual, and does SIMBAD suggest that they are genuinely uncommon sources rather than numerical leftovers?

The answer is yes. Several of the isolated spectra match SIMBAD categories such as peculiar emission-line stars, proper-motion stars, a young stellar object, and other unusual labels. Why do these classifications matter? Standard APOGEE targets are mostly cool red giants dominated by absorption lines. Detecting a peculiar emission-line star means the pipeline recognized the mathematical signature of a source with strong line behavior that does not belong to the dominant stellar population. Detecting a young stellar object

means it found a protostellar source that is physically very different from the bulk of the sample. The fact that the algorithm pushed these specific physical phenomena into the anomaly pile, without any astrophysical training, is an important validation of the full pipeline. Three representative examples are shown below.

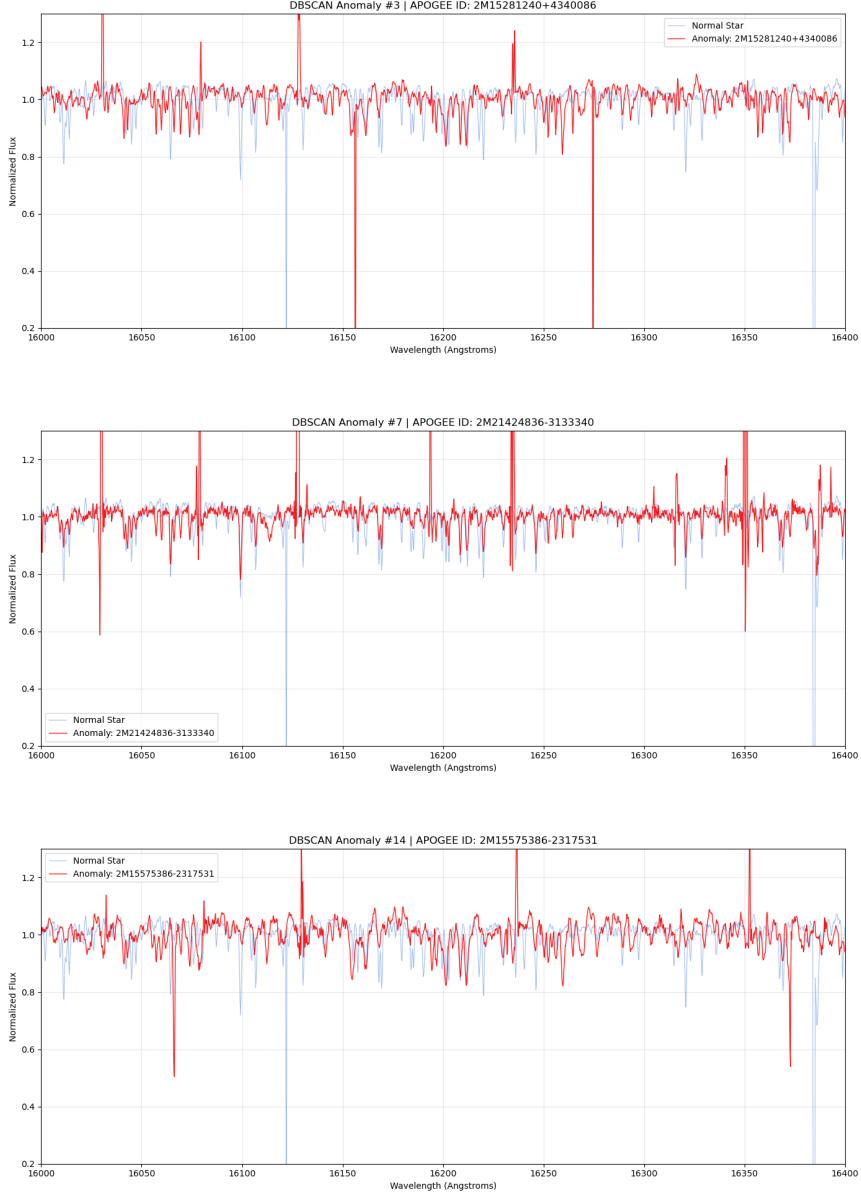


Figure 3.7: Representative anomaly spectra selected from the 25 DBSCAN outliers. These examples correspond, in SIMBAD, to a proper-motion star, a peculiar emission-line star, and a young stellar object. The comparison with a normal reference spectrum shows that the outlier set contains visibly unusual spectral behavior.

To keep the full result visible in one place, Table 3.1 lists all 25 outliers together with the SIMBAD object type and spectral type returned by the cross-match.

Table 3.1: SIMBAD summary of the 25 APOGEE outliers isolated by DBSCAN.

APOGEE ID	Object Type	Spectral Type
2M13271705-4724065	HB*	—
2M17345523-3259251	*	—
2M15281240+4340086	PM*	M3.39
2M12014432-0241176	*	—
2M06011106+5935508	PM*	M4.0V
2M05502415-5501484	*	—
2M21424836-3133340	Pe*	—
2M04571231-7108352	*	G5
2M06334130-6237090	Pe*	—
2M13331045-3302193	Pe*	—
2M19295235+1746565	RG*	—
2M05153665-6901111	*	—
2M09570267+1711061	Pe*	—
2M15575386-2317531	Y*O	M4.5
2M07570269-7204219	*	—
2M05101752-7023261	LP*	—
2M07113964-6525534	*	—
2M07261961-6517597	*	—
2M14234562-6226085	*	—
2M06030086-5847443	*	—
2M17164804-3717174	LP?	—
2M07041637-7338301	*	—
2M21390345-3043298	*	—
2M19404195+2349500	Pe*	M3.8
2M15570146-2046184	Or*	M4

The table makes the same point as the example plots, but more systematically. The outlier list is not made up of one single class. It contains a mixture of generic stars, peculiar emission-line objects, proper-motion stars, long-period variables, a young stellar object, and other uncommon labels. It is important to note, however, that 12 of the 25 isolated outliers are classified only as generic stars (*) in SIMBAD. Visual inspection suggests that some of these are probably not true astrophysical anomalies, but data artifacts such as imperfect pixel-gap interpolation or unflagged detector noise that survived the strict sample cuts. This is a normal trade-off in a fully unsupervised pipeline: the algorithm isolates mathematical outliers, and the final distinction between unusual physics and bad sensor data still requires human inspection. Even with that caveat, the diversity of the remaining labels is exactly what we would hope to see from an unsupervised anomaly search.

3.8 A Brief GALAH Trial

In addition to APOGEE, we also made a preliminary attempt to apply a similar workflow to the GALAH dataset. Because preprocessing the GALAH spectra proved difficult within the scope of this project, those results were not clean enough to support interpretation and are not used anywhere in the main analysis. We include the two plots below only to document that this direction was explored.

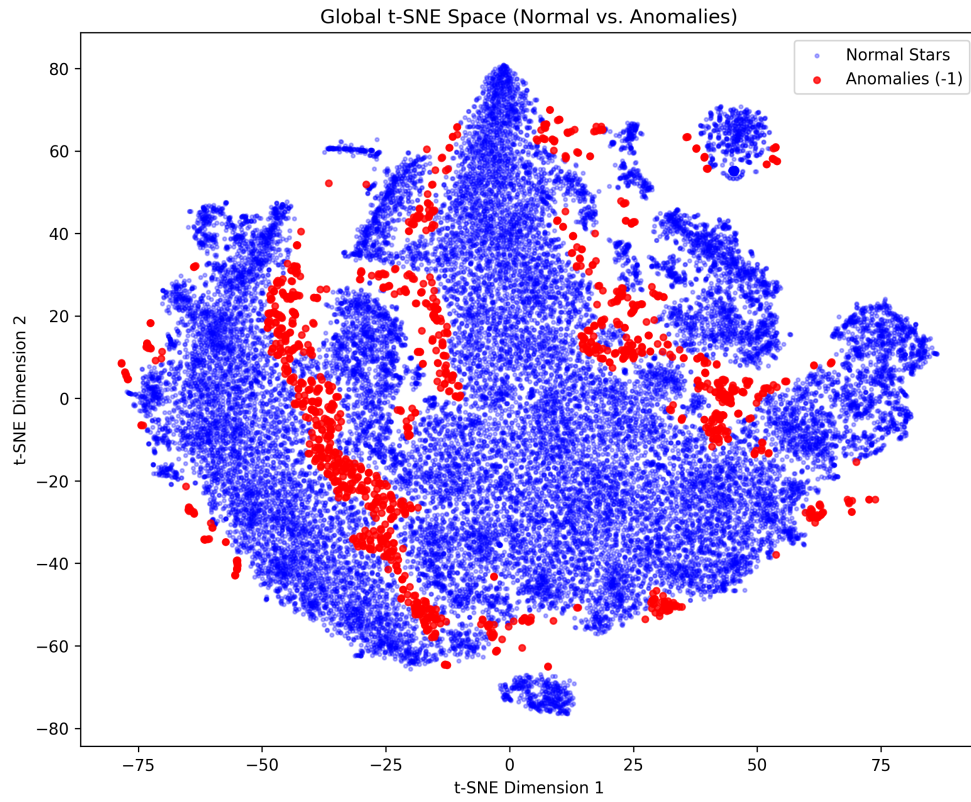


Figure 3.8: Preliminary global t-SNE map from our GALAH trial. This figure is included only as a record of the attempted analysis and is not used for interpretation in the report.

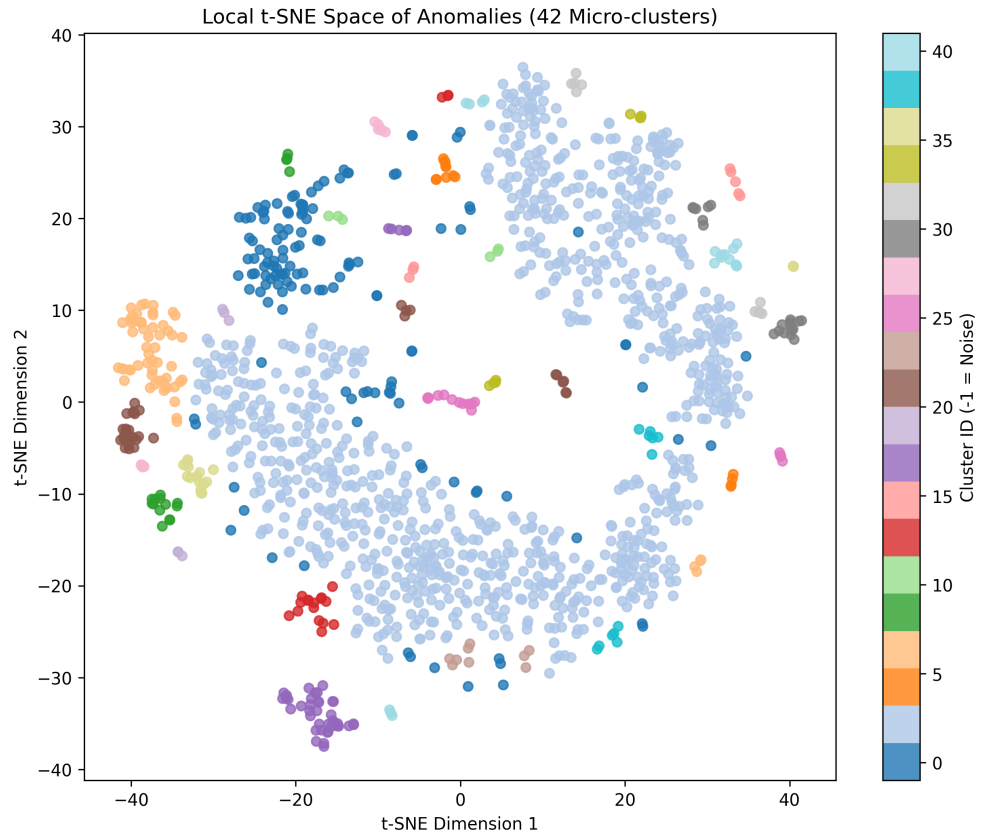


Figure 3.9: Preliminary local t-SNE map from the same GALAH trial. Because the preprocessing was not reliable enough, we do not draw conclusions from this result.

Conclusion

Part II is not an attempt to build a full astrophysical classification system. It is a demonstration that a transparent unsupervised workflow — quality filtering, careful preprocessing, PCA, t-SNE, DBSCAN, and SIMBAD follow-up — can already recover meaningful structure from APOGEE spectra and isolate a small but interesting set of anomalies.

In that sense, this report sits naturally between Traven et al. (2016) and Reis et al. (2017). From Traven et al. we take the idea that t-SNE can organize spectra visually. From Reis et al. we take the idea that large spectroscopic surveys should be searched systematically for unusual objects. Our contribution here is simpler: a smaller, clearer pipeline that still gives useful and interpretable results.

This project shows that a simple unsupervised workflow can still recover meaningful structure from very high-dimensional stellar spectra. PCA made the APOGEE spectra compact enough to handle efficiently, t-SNE organized the local spectral morphology into an interpretable map, and DBSCAN isolated a small set of unusual objects for follow-up. The SIMBAD matches suggest that many of these outliers are physically interesting sources rather than random numerical artifacts.

Overall, the report connects the mathematical idea behind neighborhood-preserving embeddings with a real astronomical application. The complete implementation used in this report is available at https://github.com/AkshayHarits/tsne_project/tree/add-phase2.

References

- [1] Geoffrey Hinton and Sam Roweis. *Stochastic Neighbor Embedding*. Advances in Neural Information Processing Systems 15, 2002.
- [2] Laurens van der Maaten and Geoffrey Hinton. *Visualizing Data using t-SNE*. *Journal of Machine Learning Research*, 9:2579–2605, 2008.
- [3] G. Traven, G. Matiječić, T. Zwitter, M. Žerjal, J. Kos, M. Asplund, J. Bland-Hawthorn, A. R. Casey, G. De Silva, K. Freeman, J. Lin, S. L. Martell, K. J. Schlesinger, S. Sharma, J. D. Simpson, D. B. Zucker, B. Anguiano, G. Da Costa, L. Duong, J. Horner, E. A. Hyde, and collaborators. *The GALAH Survey: Classification and Diagnostics with t-SNE Reduction of Spectral Information*. 2016.
- [4] Itamar Reis, Dovi Poznanski, Dalya Baron, Gail Zasowski, and Sahar Shahaf. *Detecting Outliers and Learning Complex Structures with Large Spectroscopic Surveys: A Case Study with APOGEE Stars*. *Monthly Notices of the Royal Astronomical Society*, 2017.

- [5] Sloan Digital Sky Survey. *APOGEE Survey Overview*. SDSS Technical Summary.
- [6] Sloan Digital Sky Survey. *Ways to Access APOGEE Data*. SDSS Data Access Guide.
- [7] Sloan Digital Sky Survey. *APOGEE Bitmasks*. SDSS Documentation.
- [8] Sloan Digital Sky Survey. *Datamodel: allStar Summary Catalog Documentation*. SDSS Documentation.

A. Selected Code Snippets

This appendix gives a small set of the most important code excerpts from the project. We have intentionally kept the appendix selective: each snippet is followed by a simple explanation of what it is doing in the full workflow. The complete implementation remains available at https://github.com/AkshayHarits/tsne_project/tree/add-phase2.

A.1 Core SNE/t-SNE Code

This section collects the key code ideas behind the custom SNE/t-SNE work from Part I.

Snippet A.1: Backpropagation in the custom autograd engine

Source: `autograd.py`

This is the reverse pass of the custom autograd engine. It builds the computation graph in reverse order, sends gradients backward, and then clears old references so that the large temporary arrays used in t-SNE do not stay in memory longer than needed.

```
def backward(self):
    """Performs backpropagation starting from this Value node."""
    topo = []
    visited = set()
    def build_topo(v):
        if v not in visited:
            visited.add(v)
            for parent in v._prev:
                build_topo(parent)
            topo.append(v)

    build_topo(self)

    self.grad = np.ones_like(self.data, dtype=np.float64)

    for node in reversed(topo):
        # ...

    for node in topo:
        node._backward = lambda: None
        node._prev = set()
```

Snippet A.2: Matching the target perplexity

Source: *tsne.py*

This code builds the high-dimensional probability matrix used by SNE/t-SNE. In simple terms, it adjusts the Gaussian width around each point until the number of effective neighbors matches the chosen perplexity.

```
print("Computing pairwise distances...")
(n, d) = X.shape
sum_X = np.sum(np.square(X), 1)
D = np.add(np.add(-2 * np.dot(X, X.T), sum_X).T, sum_X)
P = np.zeros((n, n))
beta = np.ones((n, 1))
logU = np.log(perplexity)

# ...

betamin = -np.inf
betamax = np.inf
Di = D[i, np.concatenate((np.r_[0:i], np.r_[i+1:n]))]
(H, thisP) = Hbeta(Di, beta[i])

Hdiff = H - logU
tries = 0
while np.abs(Hdiff) > tol and tries < 50:
    if Hdiff > 0:
        betamin = beta[i].copy()
        if betamax == np.inf or betamax == -np.inf:
            beta[i] = beta[i] * 2.
        else:
            beta[i] = (beta[i] + betamax) / 2.
    else:
        betamax = beta[i].copy()
        if betamin == np.inf or betamin == -np.inf:
            beta[i] = beta[i] / 2.
        else:
            beta[i] = (beta[i] + betamin) / 2.

    (H, thisP) = Hbeta(Di, beta[i])
    Hdiff = H - logU
    tries += 1

P[i, np.concatenate((np.r_[0:i], np.r_[i+1:n]))] = thisP
```

Snippet A.3: The main optimization step of the custom t-SNE engine

Source: *tsne.py*

This is the heart of the custom testbed. It computes pairwise distances in the 2D map, chooses the low-dimensional kernel, forms the probability matrix, computes the loss, backpropagates the gradient, and updates the embedding with momentum.

```
Y_val = Value(Y) # Wrap the current NumPy positions in autograd

# 1. Pairwise squared distances:  $D = \sum(Y^2) + \sum(Y^2).T - 2*(Y @ Y.T)$ 
Y_sq = Y_val ** 2
sum_Y = Y_sq.sum(axis=1, keepdims=True)
D = sum_Y + sum_Y.T - (Y_val @ Y_val.T) * 2.0

# -----
# --- MODIFIED: Distribution Router for Testbed ---
# -----
if dist_type == 't-distribution':
    # Parameterized Student's t-distribution
    exponent = -(nu + 1.0) / 2.0
    num = (D * (1.0 / nu) + 1.0) ** exponent

# ...

elif dist_type == 'gaussian':
    # Standard SNE (Light tails, causes crowding)
    num = (-D).exp()

elif dist_type == 'logistic':
    # Logistic SNE (Math failure state / vanishing gradients)
    num = (D.exp() + 1.0) ** -1.0

else:
    raise ValueError(f"Unknown dist_type: {dist_type}. Check orchestrator script.")
# -----

# 3. Mask out the diagonal
num_masked = num * diag_mask

# 4. Normalize to get Q
Q_val = num_masked / num_masked.sum()

# 5. Calculate Loss (Cross Entropy equivalent of KL Divergence for fixed P)
# Loss =  $-\sum(P * \log(Q))$ 
loss = -(Q_val.log() * P).sum()
```

```

# ...

# -----
if iter < 20:
    momentum = initial_momentum
else:
    momentum = final_momentum

gains = (gains + 0.2) * ((dY > 0.) != (iY > 0.)) + \
        (gains * 0.8) * ((dY > 0.) == (iY > 0.))
gains[gains < min_gain] = min_gain

iY = momentum * iY - eta * (gains * dY)
Y = Y + iY
Y = Y - np.tile(np.mean(Y, 0), (n, 1))

```

A.2 APOGEE Processing Pipeline

This section shows how the raw APOGEE survey files are converted into a clean machine-learning dataset and then mapped into two dimensions.

Snippet A.4: Selecting a clean stellar sample

Source: catalog_manager.py

This filtering step keeps only stars that satisfy the main quality conditions used in the report. The goal is to avoid filling the pipeline with low-quality spectra or obvious survey artifacts.

```

print("Applying quality filters...")
mask = (
    (df['SNR'] > 100) &
    (df['STARFLAG'] == 0) &
    (df['ASPCAPFLAG'] == 0) &
    (df['TEFF'] < 8000) &
    (df['VSCATTER'] < 1.0)
)
clean_df = df[mask].copy()

# Clean up strings (removes the b'...' formatting if present and strips spaces)
for col in ['APOGEE_ID', 'TELESCOPE', 'FIELD']:
    clean_df[col] = clean_df[col].str.replace(r"^b'|'$", "", regex=True).str.strip()

print(f"Found {len(clean_df)} perfect stars. Sampling {sample_size}...")

final_sample = clean_df.sample(n=min(sample_size, len(clean_df)), random_state=42)

```

```
final_sample.to_csv(OUTPUT_CSV, index=False)
```

Snippet A.5: Cleaning, shifting, and normalizing spectra

Source: *preprocess_spectra.py*

This preprocessing block is one of the most important parts of Part II. It masks bad pixels, rejects badly broken spectra, shifts each spectrum back toward the rest frame, interpolates across gaps, and finally normalizes the flux so that stars can be compared fairly.

```
# --- THE TRUE ASPCAP FILTER ---
# The ASPCAP pipeline marks bad pixels and camera gaps by
# driving the error extremely high or setting it to NaN.
# A 20% error (0.2) on normalized flux is our safe cutoff.
bad_pixels = (err > 0.2) | np.isnan(err) | np.isnan(flux) | (flux <= 0)

# The APOGEE camera gaps make up ~5% of the 8575 pixels.
# If more than 20% of the star is broken, we reject it entirely.
if np.sum(bad_pixels) > (len(flux) * 0.2):
    continue

flux_clean = np.copy(flux)
flux_clean[bad_pixels] = np.nan

# De-redshifting
doppler_factor = 1.0 + (v_helio / C_KM_S)
obs_wavelengths = master_wavelengths * doppler_factor

valid_mask = ~np.isnan(flux_clean)

# Interpolate over the bad pixels and camera gaps
f_interp = interp1d(
    obs_wavelengths[valid_mask],
    flux_clean[valid_mask],
    bounds_error=False,
    fill_value=1.0
)

rest_flux = f_interp(master_wavelengths)
rest_flux = np.nan_to_num(rest_flux, nan=1.0)

# --- NEW: Per-Spectrum Normalization ---
median_flux = np.median(rest_flux)
if median_flux > 0:
    rest_flux = rest_flux / median_flux
```

Snippet A.6: PCA compression and t-SNE mapping

Source: *run_pca.py* and *run_tsne.py*

This combined excerpt shows the transition from high-dimensional spectra to a 2D map. First PCA compresses the spectra to 50 components, and then t-SNE builds the non-linear embedding used for visualization and anomaly search.

```
# 1. Standardize the data (Mean centering is required for PCA)
# We subtract the mean spectrum from every star
mean_spectrum = np.mean(flux_matrix, axis=0)
centered_matrix = flux_matrix - mean_spectrum

# 2. Run PCA
# We will compress 8,575 pixels down to 50 principal components
n_components = 50
print(f"Running PCA to compress to {n_components} components...")

pca = PCA(n_components=n_components, random_state=42)
pca_features = pca.fit_transform(centered_matrix)

# 3. Calculate how much "information" we kept
variance_retained = np.sum(pca.explained_variance_ratio_) * 100
print(f"Total Variance Retained: {variance_retained:.2f}%")

# 4. Save the compressed ML-ready matrix
np.save(OUTPUT_PCA, pca_features)
print(f"Saved compressed matrix: {pca_features.shape} to {OUTPUT_PCA}")

# ...

print("Running t-SNE... (This calculates the non-linear distances)")

# --- UPDATED: Increased perplexity to 100 for large datasets and explicitly set
# max_iter ---
tsne = TSNE(n_components=2, perplexity=100, max_iter=2000, random_state=42, init='pca',
            learning_rate='auto')
X_tsne = tsne.fit_transform(X_pca)
np.save("data/tsne_2d.npy", X_tsne)

# ...

# --- NEW PLOT 3: Colored by Surface Gravity (Giant/Dwarf separation) ---
logg_dict = dict(zip(df['APOGEE_ID'].astype(str), df['LOGG']))
logg_values = np.array([logg_dict.get(str(star_id), np.nan) for star_id in valid_ids
                        ])

plt.figure(figsize=(10, 8))
# 'magma' is great for structural density
```



```
scatter3 = plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=logg_values, cmap='magma', s=25,
                      alpha=0.9)
cbar3 = plt.colorbar(scatter3)
cbar3.set_label('Surface Gravity [LOGG]')
```

A.3 Outlier Detection and Follow-up

The final stage of the project uses the t-SNE map to isolate unusual spectra and then checks whether those outliers correspond to physically meaningful objects.

Snippet A.7: Extracting anomalies with DBSCAN

Source: `run_dbscan.py`

This block applies DBSCAN to the 2D map. Dense regions become the main population, while isolated points are treated as candidate anomalies and saved for later inspection.

```
print("Running DBSCAN to find dense islands and isolate outliers...")
# eps is the maximum distance between two samples for one to be considered as in the
# neighborhood of the other.
# min_samples is the number of samples in a neighborhood for a point to be considered
# as a core point.
# You may need to tweak eps depending on how spread out your t-SNE map is!
dbscan = DBSCAN(eps=3.5, min_samples=15)
labels = dbscan.fit_predict(X_tsne)

# Calculate stats
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise = list(labels).count(-1)

print(f"Found {n_clusters} distinct clusters.")
print(f"Isolated {n_noise} anomalous stars (Noise/Outliers)!")

# ...

# --- SAVE THE OUTLIERS ---
print("Cross-referencing anomalous IDs with main catalog...")
outlier_ids = valid_ids[noise_mask]

# Filter original dataframe for just the weird stars
outlier_df = df[df['APOGEE_ID'].isin(outlier_ids)].copy()

# Save to a new CSV for the next phase of your project
outlier_df.to_csv(OUTLIER_OUTPUT, index=False)
print(f"Successfully saved {len(outlier_df)} outliers to {OUTLIER_OUTPUT}!")
```

Snippet A.8: SIMBAD cross-match for astrophysical interpretation

Source: *fetch_simbad_data.py*

This final step checks whether the mathematical outliers have meaningful astrophysical labels. The code reformats the APOGEE identifiers for SIMBAD, queries object types and spectral types, and writes the returned classifications into a compact summary table.

```
# 2. Convert APOGEE IDs (2M...) to SIMBAD format (2MASS J...)
formatted_ids = [f"2MASS J{id[2:]}" if id.startswith("2M") else id for id in ids]

# 3. Configure SIMBAD
custom_simbad = Simbad()
custom_simbad.reset_votable_fields()
# We only add the fields that aren't provided by default
custom_simbad.add_votable_fields('otype', 'sp_type', 'velocity')

# 4. Execute the Query
print("Connecting to SIMBAD servers (Strasbourg, France)...")
try:
    result_table = custom_simbad.query_objects(formatted_ids)

# ...

# 5. Convert to Pandas
simbad_df = result_table.to_pandas()

# Clean up byte strings (very common in astroquery)
for col in simbad_df.columns:
    if simbad_df[col].dtype == object:
        simbad_df[col] = simbad_df[col].apply(lambda x: x.decode('utf-8') if
        isinstance(x, bytes) else x)

# 6. Create the final summary
# SIMBAD usually returns uppercase column names. We'll use .get() to be safe.
summary_table = pd.DataFrame({
    'APOGEE_ID': ids,
    'MATCHED_ID': simbad_df.get('TYPED_ID', simbad_df.get('typed_id', 'N/A')),
    'Object_Type': simbad_df.get('OTYPE', simbad_df.get('otype', 'Unknown')),
    'Spectral_Type': simbad_df.get('SP_TYPE', simbad_df.get('sp_type', 'N/A')),
    'Radial_Vel': simbad_df.get('VELOCITY', simbad_df.get('velocity', 'N/A'))
})

# 7. Save to CSV
summary_table.to_csv(FINAL_TABLE_PATH, index=False)
print(f"\nSuccess! Summary table saved to {FINAL_TABLE_PATH}")
```